# Kızıl Gezegen Rover Kontrol Arayüzü: FAZ 3

**Eğitmen:** Melikşah Eryılmaz
**Hedef Aşama:** Hafta 3: Pointerlar & Bellek Mimarisi
**Ders Yeri:** EEM Binası - E4 Sınıfı

| BAŞLANGIÇ BATARYA | BAŞLANGIÇ KONUM | MÜHİMMAT | BELLEK MODELİ |
|---|---|---|---|
| %100.0 | (0, 0) | 5 Adet Roket | İşaretçiler (Pointers & RAM) |

## 1. Görev Senaryosu & Faz 3 Güncellemesi

Faz 1 ve Faz 2 aşamalarında kurulan tüm hareket, menzil ve operasyonel kurallar aynen devam etmektedir. Faz 3'te projenin tüm değişkenlerini yapay şekilde pointer'a boğmak yerine, işaretçi mantığı **en çok ihtiyaç duyulan ve anlamlı olduğu alanlarda** (telemetri tampon dizisi ve bellek adreslerinin doğrudan incelenmesi) kullanılacaktır:

1. **Donanımsal RAM Adresi Takibi (Telemetri Ekranı):** 10 elemanlık veri günlüğü dizisinin bellekteki ilk hücresinin fiziksel RAM adresi (`&dizi[0]` veya doğrudan dizi adı), telemetri durum raporunda doğrudan ekrana yazdırılmalıdır.
2. **Seçili Fonksiyonlarda Pointer ile Durum Yönetimi:** Yalnızca veri toplama (`cevreselAnaliz`) ve bellek temizleme (`bellekTemizle`) gibi sayaç ve tampon yönetimini ilgilendiren operasyonlarda açık pointer (`int*`, `float*`) kullanılmalıdır.
3. **Pointer ile Dizi Taraması & Kritik Ölçüm Adresi:** Ortalama hesabı pointer aritmetiğiyle yapılmalı; ayrıca kayıtlı ölçümler içerisinden en yüksek değeri bulan ve bu değerin *RAM adresini* `float*` olarak geri dönen bir analiz fonksiyonu eklenmelidir.

## 2. Telemetri Bellek Takibi & Güvenlik Kuralları

• **Tampon Başlangıç Adresi:** `durumTelemetri` fonksiyonu, 10 elemanlık dizinin başladığı bellek bölgesini (Hexadecimal formatta) raporlayarak donanımsal şeffaflık sağlamalıdır.
• **Null Pointer Güvenliği:** Pointer parametresi alan fonksiyonlar, işlem yapmadan önce adresin geçerli olduğunu (`ptr != nullptr`) doğrulamalıdır.
• **Dangling Pointer Yasağı:** Kritik veri analiz fonksiyonu, fonksiyon içinde tanımlanmış yerel değişken adresi döndürmemeli; doğrudan `main` içinde yaşayan dizinin en büyük elemanına ait adresi `float*` olarak dönmelidir.

## 3. Komut Seti ve Güncellenmiş Fonksiyon Prototipi Tablosu

Temel hareket ve savunma komutları Faz 2 sadeliğinde korunmuş, pointer entegrasyonu telemetri ve bellek fonksiyonlarına odaklanmıştır:

| Komut No | Önerilen Fonksiyon İmzası | Fonksiyonun Sorumluluğu & Açıklaması |
|---|---|---|
| **Arayüz** | `void menuyuGoster()` | Rover komut listesini ve kontrol seçeneklerini ekrana düzenli olarak yazar. |
| **Hesaplama** | `float mesafeHesapla(float x1, float y1, float x2, float y2)` | İki koordinat arasındaki Öklid mesafesini Pisagor formülü ile hesaplar ve mesafeyi döner. |
| **Komut 1** | `void durumTelemetri(...)` | Konum, batarya, roket bilgisiyle birlikte **dizinin RAM başlangıç adresini**, kayıtları ve ortalamayı basar. |
| **Komut 2** | `void hedefeGit(...)` | Hedef koordinatı alır, mesafe sınırını denetler ve konumu/bataryayı günceller (Faz 2 yapısı). |
| **Komut 3** | `void roketFirlat(...)` | Menzili denetler; başarılı ise mühimmat ve bataryayı günceller (Faz 2 yapısı). |

| Komut No | Önerilen Fonksiyon İmzası | Fonksiyonun Sorumluluğu & Açıklaması |
|---|---|---|
| **Komut 4** | `void cevreselAnaliz(float* dizi, int* aktifAdet, float& bat)` | Sensör verisini üretir; diziye kaydeder ve `aktifAdet` sayacını pointer üzerinden günceller. |
| **Komut 5** | `void bellekTemizle(int* aktifAdet)` | Aktif kayıt sayısını adres üzerinden `*aktifAdet = 0` yaparak veri tamponunu sıfırlar. |
| **Komut 6** | `void bataryayiSarjEt(float& batarya)` | Mevcut bataryayı %100 doluluğa ulaştırır. |
| **Yardımcı** | `float ortalamaHesapla(const float* dizi, int adet)` | Dizideki ilk `adet` veriyi pointer aritmetiğiyle (`*(dizi + i)`) toplayıp ortalamayı döner. |
| **Analiz** | `float* kritikVeriAdresi(float* dizi, int adet)` | Kayıtlı veriler içindeki en yüksek ölçümün **RAM hücresi adresini** (`float*`) döner. |

## 4. Beklenen Konsol Arayüzü Formatı

```
KIZIL GEZEGEN - ROVER KONTROL MERKEZI (FAZ-3 : TELEMETRI BELLEK RAPORU)
 [1] Durum ve Telemetri Bilgisi (RAM Adresi & Ekstrem Analiz Dahil) [2] Hedef Koordinata Git (Maks. 50 km) [3] Roket Firlat (10 km - 100 km) [4] Cevresel Analiz Yap ve Belleğe Kaydet (Sensör Paketi) [5] Telemetri Belleğini Temizle [6] Bataryayi Sarj Et (%100) [0] Sistemi Kapat ve Cikis ---------------------------------------------------------------- Komut Seciniz: 1 ================ ROVER TELEMETRI RAPORU ================ Batarya Durumu : %86.0 Mevcut Konum : (30, 40) Kalan Roket : 4 Adet Toplam Yol : 50.00 km Tampon RAM Baslangic : [ 0x7ffd52ab41c0 ] <-- (Dizinin RAM Adresi) Bellek Doluluk : [ 3 / 10 ] Kayitli Veriler : [ 45.2, 88.6, 21.4 ] Telemetri Ortalamasi : 51.73 mSv En Kritik Olcum : 88.6 mSv [RAM Hucre Adresi: 0x7ffd52ab41c8] ==========================================================
```

```
[VERI KAYDI VE BELLEK SIFIRLAMA AKISI]
 Komut Seciniz: 4 > Olculen Radyasyon Degeri : 94.2 mSv > Veri telemetri bellegine kaydedildi (Slot: 4 / 10). Tuketilen Sarj: %2.0 Komut Seciniz: 5 > Telemetri bellegi sifirlandi (*aktifAdet = 0). Kayit durumu: 0 / 10
```

## 5. Teknik Kurallar ve Tavsiyeler

• **Tampon Adresinin Yazdırılması:** Telemetri raporunda `cout << dizi;` veya `cout << &dizi[0];` yazılarak dizinin bellekteki başlangıç adresi öğrencilere doğrudan gösterilmelidir.
• **Dizi İndeksleri & Aritmetik:** Ortalama hesabı ve en büyük elemanı bulma işlemlerinde `dizi[i]` yerine `*(dizi + i)` pointer gösterimi uygulanmalıdır.
• **Kütüphaneler:** Matematik fonksiyonları için `<cmath>`; rastgele veri üretimi için `<cstdlib>` ve `<ctime>` kütüphanelerinden yararlanınız.

**! Faz 2'deki temiz kodunuzu bozmadan, sadece telemetri dizisine ve sayaçlarına odaklanarak pointer entegrasyonunu tamamlayınız.**

**! İçerikte verilen örnek kod çıktıları sadece yönlendirme amaçlı genel çıktılardır. Kendinizce geliştirmeleri eklemekten lütfen çekinmeyin :)**

**! Olabildiğince az sayıda GLOBAL PARAMETRE kullanmaya çalışınız. Ağırlıklı olarak local tanımlı değerleri - gerekiyorsa referanslarını kullanarak iletmeye çalışınız.**